github.com / **Sentiment_Predict**

Technical Report · v1.0 · 2026-10-28

Twitter Sentiment Analysis with a Transformer Built from Scratch

Transformer from Scratch vs. Pretrained LLMs

A complete report on the project's architecture, data, training, evaluation, Streamlit application, repository audit and development roadmap — based on direct inspection of the code, notebook, saved model and data in the repository.

Python **3.11 (devcontainer)** | TensorFlow **2.17.0** | Keras **3.5.0 (pinned)** | Streamlit **≥ 1.35**

Task **Binary NLP** | License **MIT (per README)**

93.4%
Accuracy on 542 test tweets (notebook output)

39,978
Cleaned training tweets

≈2.87M
Trainable parameters (computed from code)

1 block
Transformer encoder, 4 attention heads

Project author: Officiall · Mahan Liaghatmand

Table of Contents

01	Executive Summary	3
02	Project Overview and Goals	4
03	Repository Structure	5
04	Data and Preprocessing	6
05	Model Architecture	7
06	Training Process	9
07	Evaluation and Results	10
08	Streamlit Application and Deployment	12
09	How to Run	13
10	Technical Audit: Findings and Risks	14
11	Development Roadmap (Proposed)	15
12	Appendix: Quick Reference and Glossary	17

How this report was prepared

All figures and technical claims come from three sources: (1) the outputs saved in the notebook, (2) the code in `app.py` and the configuration stored inside the `.keras` model file, and (3) direct computation on the data file. Wherever a number or claim in `README.md` does not match these sources, it is reported explicitly in the Technical Audit section. Items that are recommendations or estimates (not measured results) are labelled "proposed" or "estimated".

01 Executive Summary

Sentiment_Predict is a binary sentiment classifier for tweets (Positive / Negative). Its core is a **single-block Transformer encoder** written entirely in Keras with no pretrained weights (positional embedding, multi-head self-attention, layer normalization, feed-forward network). The model is benchmarked against the pretrained **BERT-Base** and **DistilBERT**, and a **Streamlit** GUI provides single and batch (CSV) inference.

93.36%	0.933	49% / 45%	21 min
Scratch-model accuracy on the test set (506 of 542)	Positive-class F1 (computed from the confusion matrix)	Accuracy without fine-tuning: BERT / DistilBERT (near chance)	Total training time (5 epochs on Colab)

Key achievements

- A hands-on, readable implementation of Transformer components: token + positional embedding, multi-head self-attention, residual + LayerNorm and FFN.
- A small model (≈2.87 million parameters) reaches 93.4% accuracy on the target data; the 95% Wilson confidence interval for this accuracy is roughly 90.9% – 95.2%.
- A Streamlit app with two modes (single / batch), model caching, error handling and two weight-loading strategies.
- A `.devcontainer` configuration for one-click launch in GitHub Codespaces (port 8501).

Important notes for the reader

1) The comparison with BERT/DistilBERT is not like-for-like
In the notebook these models were evaluated with a **random, untrained classification head** (load report: `MISSING: classifier.weight/bias`). The ~50% result therefore does not show a weakness of these models; it shows they are unusable without fine-tuning.

2) The README results table differs from the notebook's actual output
For example, the README reports 93.7% accuracy for the scratch model and 92.8% recall for BERT, while the saved notebook output shows 93.36% and 5.07% respectively. Details are in Section 10; a fix is proposed in the roadmap (Phase 1). In this report the **notebook values are the reference**.

02 Project Overview and Goals

The problem

Sentiment analysis on social media supports brand monitoring, product feedback and public-opinion analysis. In this project the input is a short text (a tweet) and the output is one of two labels — positive or negative — together with a confidence score.

Project goals (per README and code)

1. **Educational:** understand the internals of a Transformer by building each component by hand instead of using a ready-made backbone.
2. **Comparative:** measure how a small task-specific model stands against large pretrained models.
3. **Product:** provide a GUI for interactive use and batch processing without writing code.

Technology stack

Area	Tool	Role in the project
Modeling	`TensorFlow 2.17 / Keras 3`	Custom layers, training and model saving
Baselines	`PyTorch + Hugging Face Transformers`	Loading BERT/DistilBERT and extracting logits
Evaluation	`scikit-learn`	Data split, accuracy/precision/recall, confusion matrix
Data	`pandas, csv`	Reading, cleaning and labelling
User interface	`Streamlit ≥ 1.35`	Single and batch prediction, result download
Dev environment	`devcontainer (Python 3.11)`	Automatic app launch in Codespaces
Training	Google Colab	The notebook was run with `drive.mount`

03 Repository Structure

The actual layout of the archive (`Sentiment_Predict-main.zip`) is shown below. The repository is flat — the `notebooks/` and `app/` folders mentioned in the README do not exist.

```
Sentiment_Predict-main/
├── .devcontainer/
│   └── devcontainer.json              # Codespaces: Python 3.11, port 8501      (1.0 KB)
├── data/
│   └── twitter_training_clean_binary.csv   # 39,978 rows: text, sentiment, label (4.9 MB)
├── model/
│   ├── model_scratch_sentiment.keras  # config + weights (+ optimizer state)   (34.5 MB)
│   └── vectorizer_vocab.pkl           # 20,000-token vocabulary                 (1.4 MB)
├── Main_twitter_sentiment_binary.ipynb # data prep · training · evaluation       (98.7 KB)
├── app.py                             # Streamlit GUI                           (13.6 KB)
├── requirements.txt                   # app dependencies (6 lines)
└── README.md                          # project description                    (6.1 KB)
```

Role of each file

File	Description
`Main_twitter_sentiment_binary.ipynb`	Full pipeline: load data from Google Drive, train/val split, build `TextVectorization`, define and train the model, load BERT/DistilBERT, evaluate on test, save model and vocabulary.
`app.py`	Re-declares the custom layers, builds the architecture, loads weights and vocabulary, and serves the Streamlit UI.
`model/model_scratch_sentiment.keras`	Keras archive with `config.json`, `metadata.json` and `model.weights.h5` (saved with Keras 3.13.2).
`model/vectorizer_vocab.pkl`	The exact token list (20,000 entries, including `''` and `[UNK]`) used to rebuild the same word→index mapping as at training time.
`data/twitter_training_clean_binary.csv`	Cleaned data with columns `text, sentiment, label` (0 = Negative, 1 = Positive).
`.devcontainer/devcontainer.json`	Installs dependencies and auto-runs `streamlit run app.py`.

04 Data and Preprocessing

Dataset profile (computed directly from the CSV)

Metric	Value
Total training samples (after cleaning)	39,978
Negative (label = 0)	21,043 (52.6%)
Positive (label = 1)	18,935 (47.4%)
Missing values / exact duplicate texts	0 / 0
Tweet length (words): mean / median / max	19.6 / 16 / 189
Tweets longer than `SEQ_LEN = 40` words (truncated)	11.2%

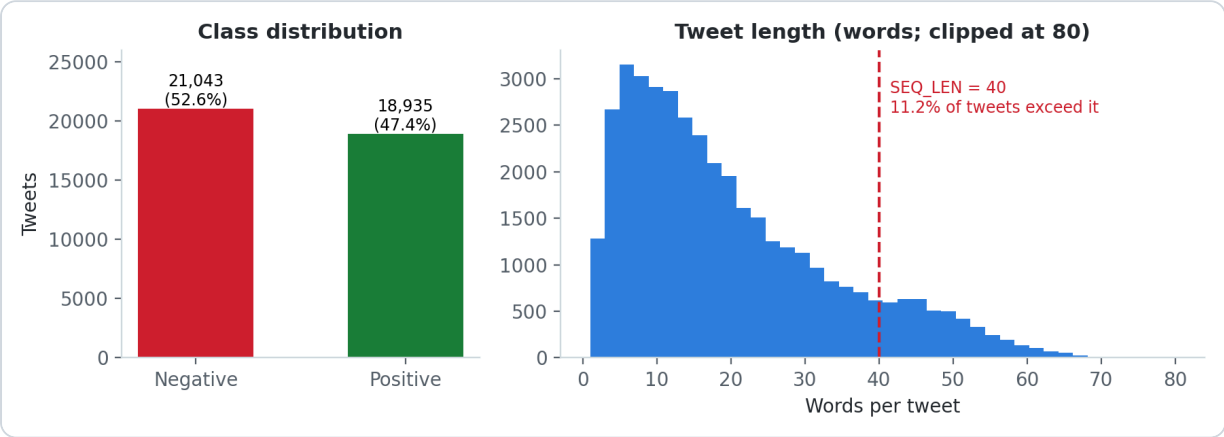


Figure 1 — Class distribution (left) and tweet-length distribution (right), computed on the data file.

The two classes are roughly balanced (≈ 1.11 : 1), so accuracy is a meaningful metric, although reporting precision/recall/F1 alongside it is recommended.

Processing steps in the notebook

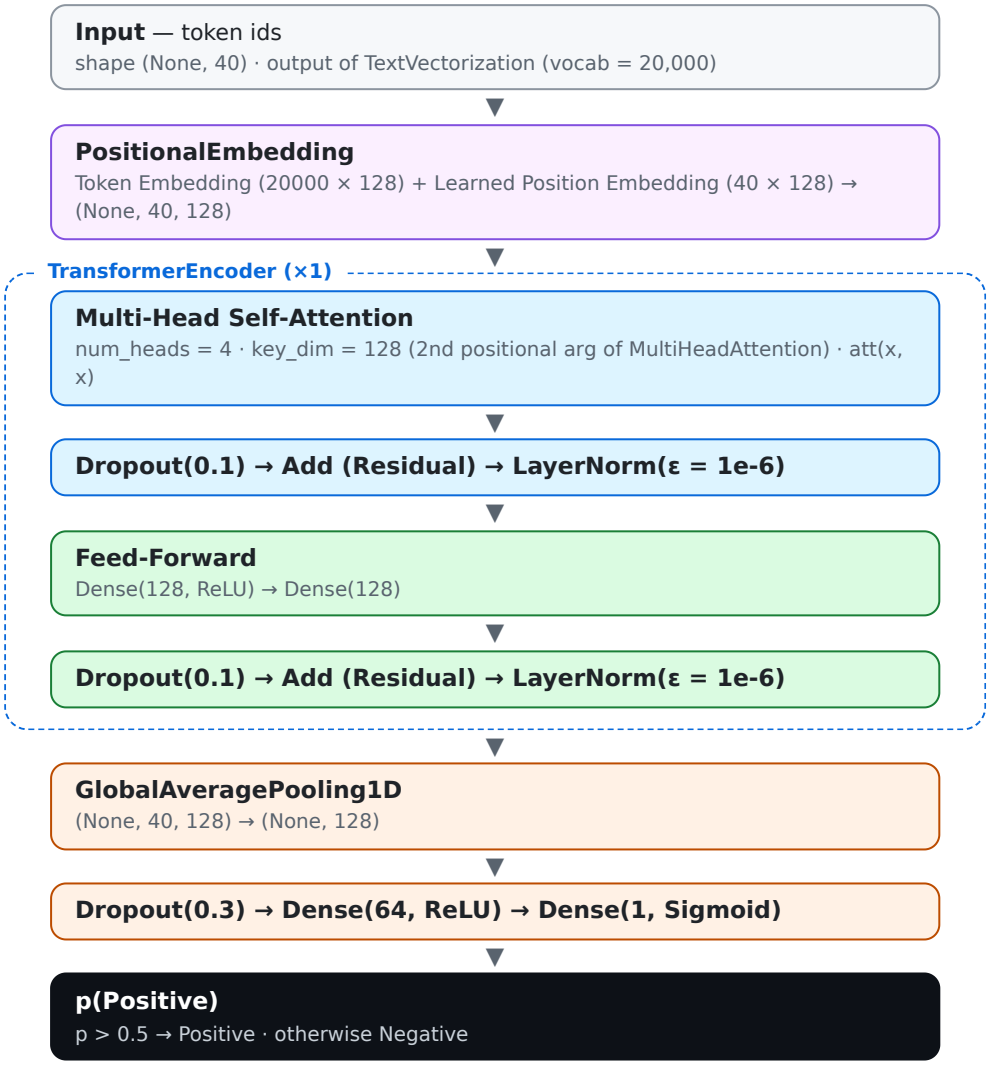
1. **Read:** `load_training` reads the file with the `csv` module (text from column 0, label from column 1).
2. **Filter labels:** only `Negative → 0` and `Positive → 1` are kept (`prep`). In the recorded run the input file already contained only these two labels (plus the header row): 39,979 raw rows → 39,978 after filtering.
3. **Split:** 90% train / 10% validation with `stratify=label`, `random_state=42` → **35,980 / 3,998**.
4. **Separate test set:** `twitter_validation.csv` with the same filter → **542** samples (266 Negative, 276 Positive).
5. **Tokenization:** `TextVectorization(max_tokens=20000, output_sequence_length=40)` with Keras defaults (lowercasing, punctuation stripping), adapted on training data only.

Data source

The structure and labels resemble the well-known "Twitter Entity Sentiment Analysis" dataset, but the README does not name the source. The raw files `twitter_training.csv` and `twitter_validation.csv` are not in the repository; only the cleaned training file is.

05 Model Architecture

The model is a single-block Transformer encoder for sequence classification. The configuration stored inside the `.keras` file (`vocab_size=20000, seq_len=40, embed_dim=128, num_heads=4, ff_dim=128, dropout=0.1`) was verified to match `app.py` and the notebook.



Hyperparameters

Parameter	Value	Parameter	Value
Sequence length	40	Optimizer	Adam (default lr)
Embedding dimension	128	Loss	Binary cross-entropy
Attention heads	4	Batch size	32
FFN dimension	128	Max epochs	15
Dropout (block / head)	0.1 / 0.3	Vocabulary size	20,000

Parameter count (computed from the code)

Component	Calculation	Parameters
Token Embedding	20,000 × 128	2,560,000
Positional Embedding	40 × 128	5,120
Attention — Q, K, V	3 × (128×512 + 512)	198,144
Attention — output projection	512×128 + 128	65,664
FFN (two Dense layers)	2 × (128×128 + 128)	33,024
LayerNorm × 2	2 × 256	512
Dense(64) + Dense(1)	8,256 + 65	8,321
Total	**—**	**2,870,785**

About 89% of parameters sit in the embedding tables. The ≈ 34.5 MB file size matches 2.87M × 4 bytes × 3 (weights + two Adam moment buffers), i.e. the file also contains optimizer state.

Architecture note: no padding mask is applied

The Embedding layer is built with `mask_zero=False` (default) and `att(x, x)` is called without an `attention_mask`. With a median tweet length of 16 words and a fixed length of 40, many samples are mostly padding, which still takes part in attention and in `GlobalAveragePooling1D`. The model reaches good results regardless, but masking is one of the simplest improvements (roadmap, Phase 4).

06 Training Process

Configuration

- `tf.data` pipeline; shuffle with `seed=42`, batch = 32, `prefetch(AUTOTUNE)`; tokenization happens inside the pipeline via `.map`.
- `EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)`.
- Steps per epoch: **1,125** (= ⌈35,980 / 32⌉).

Per-epoch results (copied from notebook output)

Epoch	Train Acc	Train Loss	Val Acc	Val Loss	Time
1	0.7585	0.5009	0.8224	0.4187	272 s
2 ★	**0.8649**	**0.3483**	**0.8339**	**0.4147**	**253 s**
3	0.8900	0.2821	0.8402	0.4208	255 s
4	0.9000	0.2450	0.8422	0.5085	253 s
5	0.9094	0.2179	0.8252	0.5165	254 s

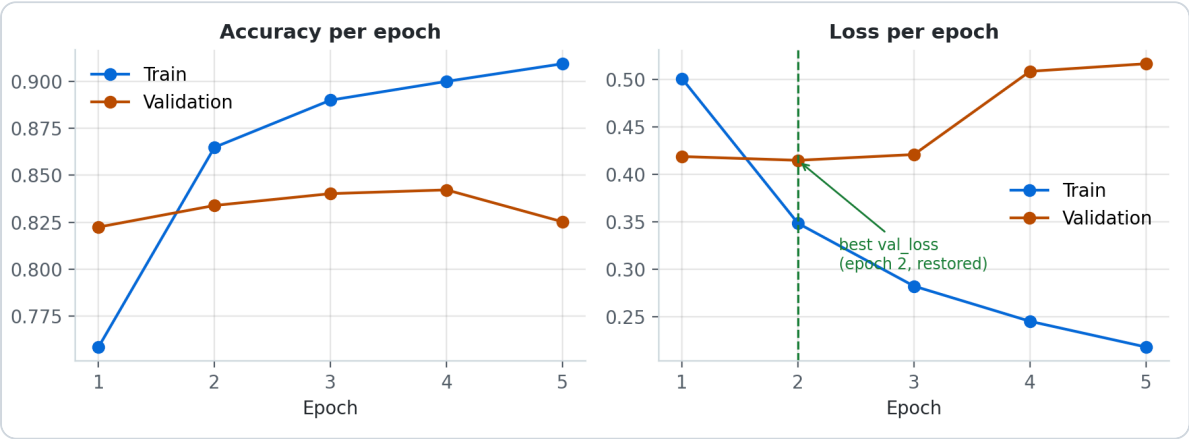


Figure 2 — Training curves. The lowest val_loss occurred at epoch 2 and those weights were restored.

Analysis

- **Early stopping:** the lowest `val_loss = 0.4147` was recorded at epoch 2; with `patience=3` training stopped after epoch 5 and the epoch-2 weights were restored. The saved model therefore holds the epoch-2 weights.
- **Overfitting:** the gap between train accuracy (90.9%) and validation accuracy (82.5%) at epoch 5, and the rising val_loss from epoch 3, indicate fast overfitting — expected for a large embedding table relative to ~36K samples.
- **Speed:** ≈ 225–237 ms/step for a small model suggests training ran on a CPU or weak accelerator (≈ 21.5 minutes in total).

> **ReduceLROnPlateau is defined but never used**
> The variable `reduce` is created in the notebook, but only `callbacks=[early]` is passed to `fit`. The README lists this callback among the training features.

07 Evaluation and Results

All three models were evaluated on one shared test set of **542** tweets (266 Negative / 276 Positive), using a 0.5 threshold for the Keras model and `argmax` over logits for the Hugging Face models.

Model	Accuracy	Precision (Pos)	Recall (Pos)	F1 (Pos)	Note
Scratch Transformer	**93.36%**	**96.15%**	**90.58%**	**93.28%**	**Trained on the target data**
BERT-Base (uncased)	49.08%	50.00%	5.07%	9.21%	No fine-tuning; random head
DistilBERT-Base	45.02%	47.43%	73.55%	57.67%	No fine-tuning; random head

Accuracy/precision/recall are taken directly from the notebook output; Positive-class F1 is computed from the confusion matrices.

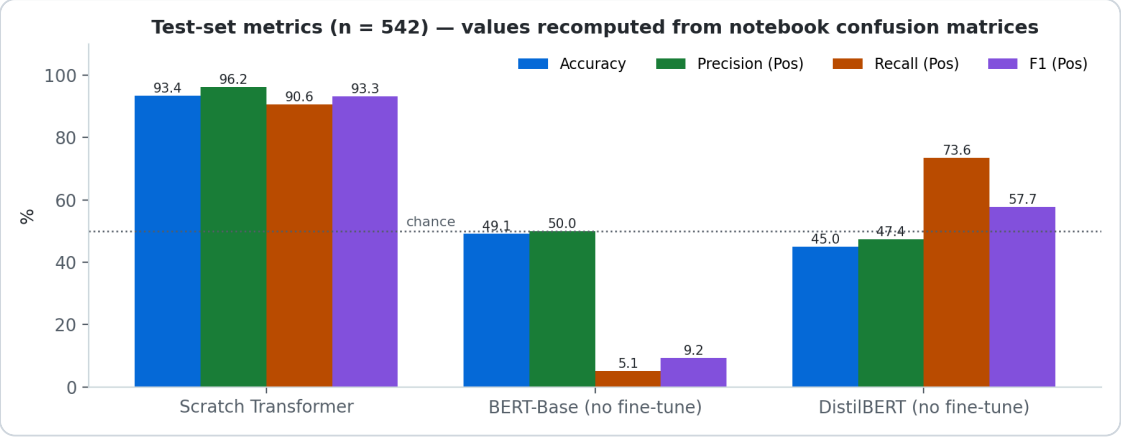


Figure 3 — Metric comparison on the test set (n = 542).

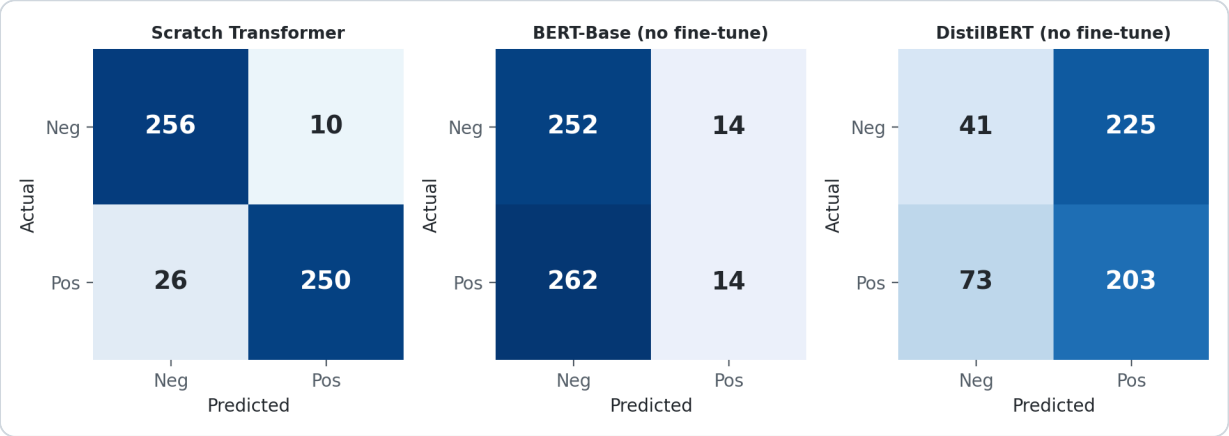


Figure 4 — Confusion matrices (rows = actual label, columns = prediction).

Per-class report for the scratch model

Class	Precision	Recall	F1	Support
Negative	0.91	0.96	0.93	266
Positive	0.96	0.91	0.93	276
Macro avg	0.93	0.93	0.93	542

- **Errors:** 10 false positives and 26 false negatives; the model is slightly conservative and labels some positive tweets as negative (Positive recall 90.6% vs. Negative recall 96.2%).
- **Statistical uncertainty:** with 542 samples, the 95% Wilson interval for accuracy is about 90.9% – 95.2%; differences under ~2 points between experiments should not be considered meaningful.
- **Validation vs. test gap:** validation accuracy is 82–84% while test accuracy is 93.4%. The cause is not documented in the repository; hypotheses to check: different distribution or label noise in the test file, the small test size, or a random split on data containing near-duplicate tweets. This should be examined in the error analysis (roadmap, Phase 2).

Why are BERT and DistilBERT near chance?

In the notebook's load log, `classifier.weight` and `classifier.bias` (and `pre_classifier` for DistilBERT) are marked `MISSING` and were newly initialized. The backbone has linguistic knowledge, but the decision layer was never trained on this task, so the output is effectively random/biased (BERT predicts almost everything Negative, DistilBERT mostly Positive). The comparison is therefore "small task-specific model vs. pretrained models with no adaptation" and says nothing about the overall superiority of either family; the README makes this same caveat correctly.

Because the head is random, every run yields different values; this may be one reason the README numbers differ from the notebook output (a hypothesis — not provable from the repository data).

08 Streamlit Application and Deployment

Prediction flow

Input Tweet text or the `text` column of a CSV ▶ **strip** Trim leading/trailing whitespace ▶ **TextVectorization** vocab from pkl · length 40 ▶ **model.predict** Sigmoid output ▶ **Decision** p > 0.5 → Positive · confidence = max(p, 1−p)

Features

- **Single analysis:** three ready examples, a green/red result card, a confidence bar and the raw Positive-class probability.
- **Batch analysis:** upload a CSV with a `text` column → adds `sentiment` and `confidence` columns (4 decimals) → shows the table and offers `sentiment_results.csv` for download.
- **Sidebar:** live architecture and vocabulary-size information.
- **Robustness:** `@st.cache_resource` loads the model once; load errors are shown in the UI (instead of crashing) with a hint about TF/Keras version mismatch.

Model loading: two sequential strategies

Step	Method	Reason
1	Rebuild the architecture in code + `model.load_weights()`	Independent of how custom layers were serialized
2 (fallback)	`load_model(custom_objects=…, safe_mode=False)`	Compatibility with different Keras versions

The vocabulary design is also notable: instead of rebuilding the vectorizer from the CSV (which silently corrupts the word→index mapping if order or preprocessing differs), the exact token list is read from `vectorizer_vocab.pkl`. This is the soundest engineering decision in the app, and the README's description of "rebuilding from the CSV" no longer matches the current code.

Runtime environment

- **Codespaces:** Python 3.11 image; after creation it runs `pip install -r requirements.txt` and then `streamlit run app.py --server.enableCORS false --server.enableXsrfProtection false`; port 8501 opens in Preview.
- **Dependencies:** `streamlit≥1.35, tensorflow==2.17.0, keras==3.5.0, pandas≥2.0, numpy≥1.26,<2.0, scikit-learn≥1.3`. (`scikit-learn` is not used in `app.py`.)

Security warnings

Both `pickle.load` and `load_model(..., safe_mode=False)` can execute arbitrary code while loading. Only trusted files (this repository) should be loaded. Also, disabling CORS and XSRF in the devcontainer suits Codespaces, not public deployment.

09 How to Run

Run the app locally

```
git clone https://github.com/<your-username>/Sentiment_Predict.git
cd Sentiment_Predict

python -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -r requirements.txt
streamlit run app.py                                   # → http://localhost:8501
```

> The repository is flat; the `cd app` step mentioned in the README is not needed. TensorFlow 2.17 supports Python 3.9–3.12; it may fail to install on newer Python versions.

Reproduce training

```
pip install tensorflow==2.17.0 torch transformers scikit-learn pandas matplotlib
jupyter notebook Main_twitter_sentiment_binary.ipynb
```

The notebook was written for Google Colab (`drive.mount` and `/content/drive/MyDrive/…` paths) and requires the raw files `twitter_training.csv` and `twitter_validation.csv`, which are not in the repository. For local use, fix the paths and supply the files.

Batch use

```
# input.csv
text
"I absolutely love this game, best purchase ever!"
"This update ruined everything, so disappointed."
```

The download contains the columns `text, sentiment, confidence`.

10 Technical Audit: Findings and Risks

Findings are based on direct inspection of the files. Severity: High Medium Low Info

#	Severity	Finding and evidence	Recommendation
1	High	**README numbers ≠ notebook output** (Acc / Prec / Rec). Scratch: README 93.7 / 95.2 / 92.4 ↔ notebook 93.36 / 96.15 / 90.58 BERT: README 50.0 / 50.5 / 92.8 ↔ notebook 49.08 / 50.00 / 5.07 DistilBERT: README 53.3 / 52.4 / 92.8 ↔ notebook 45.02 / 47.43 / 73.55	Save results to `results.json` with fixed seeds and generate the README table from it.
2	Medium	**README structure and description are outdated:** `notebooks/` and `app/` folders do not exist; README says the vectorizer is rebuilt from the CSV but `app.py` reads `vectorizer_vocab.pkl`; it references `app/README.md`, which is not in the repository.	Update the README to match the real structure.
3	Medium	**ReduceLROnPlateau unused** (only `early` is passed to `fit`) yet listed in the README.	Add it to `callbacks` or remove it from the README.
4	Medium	**Version mismatch:** the model was saved with Keras 3.13.2 while `requirements.txt` pins `keras==3.5.0`. The fallback in `app.py` reduces but does not remove this risk.	Train/save and run with one version; add a load smoke test.
5	Medium	**Pickle portability:** the vocab file contains `np.str_` objects referencing `numpy._core` (NumPy 2.x) while `requirements.txt` restricts `numpy<2.0`. Loading may fail (not executed in this report; needs verification).	Store the vocabulary as plain text or JSON.
6	Medium	**BERT/DistilBERT compared without fine-tuning** and non-deterministic (random head); says nothing about their real quality.	Fair fine-tuning (roadmap Phase 3).
7	Medium	**No padding mask** in Embedding, Attention or Pooling (Section 5).	`mask_zero=True`, attention mask and masked pooling.
8	Low	**Reproducibility:** the notebook depends on Google Drive and raw files outside the repo; `SettingWithCopyWarning` in `prep`; a Persian "replace with your path" comment remains in the code.	Parameterized paths, a `train.py` script, recorded seeds.
9	Low	**Second `adapt` call** in the last notebook cell before saving the vocab (on the same data). Usually identical, but exporting right after the first adapt is safer.	Export the vocab in the same cell that builds the vectorizer.
10	Low	**Model size:** 34.5 MB because optimizer state is saved; weights alone ≈ 11.5 MB (estimate).	`model.save(..., include_optimizer=False)`; use Releases/Git LFS for larger models.
11	Low	**License:** the README declares MIT but there is no `LICENSE` file in the archive.	Add a `LICENSE` file and state the data source/license.
12	Info	**Practical limits:** only two classes (neutral text is forced to positive/negative); "confidence" is just `max(p, 1−p)` and is uncalibrated; gaming-domain data (e.g. "murder you all" labelled Positive) suggests sensitivity to domain slang; no tweet-specific preprocessing (URLs, mentions, emoji).	Calibration, a Neutral class, domain analysis (Phases 2 and 4).

11 Development Roadmap (Proposed)

The roadmap below is **proposed**, built from the audit findings and the "Future Work" section of the README; the timeframes are rough estimates for a single developer, not commitments.



Current status (v1.0): ✔ scratch model · ✔ comparison with BERT/DistilBERT (no fine-tuning) · ✔ Streamlit GUI · ✔ devcontainer.

Phase 1 — Stabilize and document

Priority: high · weeks 1–2

- Fix the README results table and folder structure; add `LICENSE`, data source/license and `.gitignore`.
- Add `ReduceLROnPlateau` to training (or remove it from the docs); record seeds.
- Save the vocabulary as text/JSON; save the model with `include_optimizer=False`; align TF/Keras versions.
- Store metrics in `results.json` and generate the README table from it; add a Model Card.

Success criteria: every README number equals `results.json`; the app runs on a clean environment with a single command.

Phase 2 — Rigorous evaluation and error analysis

Priority: high · weeks 3–4

- Report F1, ROC-AUC, PR-AUC and confidence intervals; repeat training with several seeds or k-fold.
- Investigate the validation (≈83%) vs. test (≈93%) gap: remove near-duplicates, check for leakage, analyze error samples.
- Calibrate probabilities (temperature scaling) and choose an optimal threshold instead of a fixed 0.5.
- Cheap classical baseline: TF-IDF + Logistic Regression to gauge the real value of the Transformer.

Success criteria: an evaluation report with mean ± std over multiple runs and a list of the most common error categories.

Phase 3 — Fair fine-tuning of pretrained models

Priority: high · weeks 5–7

- Fine-tune DistilBERT and BERT on the same split (e.g. 2–3 epochs, learning rate ≈ 2e-5, `max_length` ≥ 40).
- Compare accuracy/F1, parameter count, file size and inference latency (ms/tweet) in one table.
- Replace the current README comparison with this fair result.

Success criteria: a three-axis table (accuracy · size · speed) and a documented recommendation for choosing a model by use case.

Phase 4 — Model improvements and problem extension Priority: medium · weeks 8–10

- Padding mask (`mask_zero=True` + `attention_mask`) and masked pooling.
- Subword tokenizer (WordPiece/BPE) to reduce OOV; try pretrained embeddings; more encoder blocks.
- Reduce overfitting: embedding dropout, label smoothing, weight decay.
- Return to the 4-class problem (Positive / Neutral / Negative / Irrelevant) and add tweet-specific preprocessing (URLs, mentions, emoji).

Success criteria: a measurable improvement (beyond the confidence interval) over the Phase 2 baseline, with a documented ablation.

Phase 5 — Productization and deployment Priority: medium · weeks 9–12

- Unit tests (sample predictions, model loading) and GitHub Actions: lint + test + app smoke test.
- Dockerfile, deployment to Streamlit Community Cloud or Hugging Face Spaces, and an API (e.g. FastAPI) for integration.
- ONNX / TFLite export with latency measurement; model versioning via GitHub Releases or Git LFS.
- UI upgrades: explainability (e.g. word importance), prediction history, distribution chart in batch mode.

Success criteria: green CI pipeline, a public demo with a stable link, and an optimized model with documented latency.

Main risks and mitigations

Risk	Impact	Mitigation
TF/Keras or NumPy version incompatibility across environments	Model/vocab load failure	Consistent pins, text format for vocab, smoke test in CI
Noisy labels and gaming-domain bias	Weak generalization to other domains	Diverse test sets, error analysis
Small test set (542 samples)	±2% uncertainty	Confidence intervals, multiple seeds, a larger test set
Loading pickle files / `safe_mode=False`	Arbitrary code execution with a malicious file	Safe formats, load only from trusted sources

12 Appendix: Quick Reference and Glossary

Quick reference

Item	Value
Saved model name	scratch_transformer (Functional)
Input / output	(None, 40) int → (None, 1) sigmoid
Labels	0 = Negative · 1 = Positive
Data split	35,980 / 3,998 / 542
Best epoch (val_loss)	2 (0.4147)
Test accuracy / 95% CI	93.36% / 90.9–95.2%
Model / vocab / data file size	34.5 MB / 1.4 MB / 4.9 MB
App port	8501

Glossary

Term	Meaning
Self-Attention	A mechanism that represents each token by weighting all other tokens in the sequence.
Positional Embedding	A learned vector per position that gives the model word order.
Residual + LayerNorm	Adding a sublayer's input to its output, then normalizing, for training stability.
Fine-tuning	Continuing to train a pretrained model on the target task's data.
OOV	Words not in the vocabulary, mapped to `[UNK]`.
Precision / Recall / F1	Correctness of positive predictions / coverage of true positives / harmonic mean of the two.
Calibration	Aligning model output probabilities with the true frequency of being correct.

Conclusion

The project is a good educational and practical example, with a strong result for a small task-specific model. By fixing the documentation inconsistencies, adding a fair fine-tuning comparison and a more rigorous evaluation, it can become a reliable reference repository for sentiment analysis.